Sparse geometric representation of images using Gaussian splatting

(Bachelor thesis)

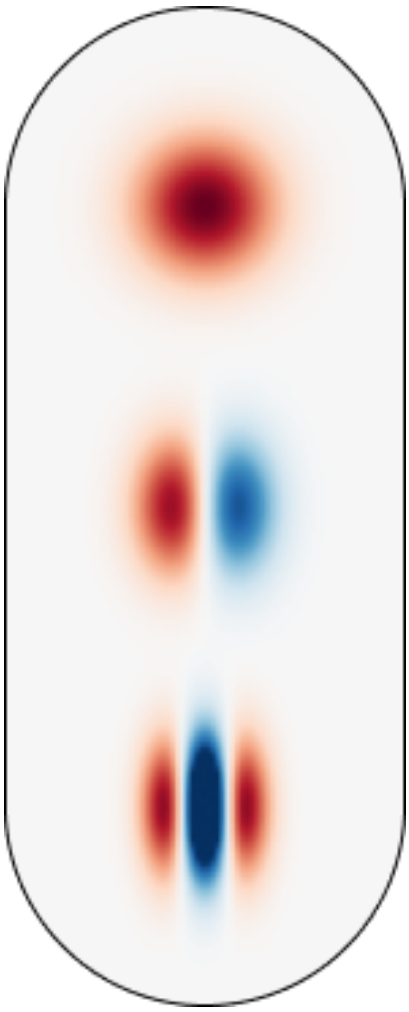
Victor Klomp

December 10, 2025

Technical university of Eindhoven

Bart M.N. Smets
(Supervisor)

Finn M. Sherry
(Supervisor)



Contents

1 Introduction . . . 3
2 Theory . . . 4
2.1 Gaussian layers . . . 4
2.2 Lie groups . . . 5
2.2.1 Affine group . . . 5
2.2.2 Lie groups & parameter space . . . 7
2.2.3 The Lie algebra $\mathfrak{aff}(\mathbb{R}^2)$ . . . 7
2.2.4 The exponential & logarithmic map . . . 8
2.3 Kaji-Ochiai parameterization . . . 8
2.4 A note on shearing . . . 9
3 Methods . . . 12
3.1 Rendering function . . . 12
3.2 Loss . . . 13
3.3 Culling . . . 13
4 Results . . . 13
4.1 Initialization . . . 14
4.2 Artifacts . . . 15
4.3 Number of Gaussians . . . 17
4.4 Culling . . . 18
4.5 Higher order Gaussians . . . 19
5 Conclusion . . . 20
6 Future work . . . 20
Bibliography . . . 22

1 Introduction

In the last few years, AI and machine learning have become increasingly important. We have seen ML applied across several domains, mainly text, image, video, and audio. For image recognition, convolutional neural networks (CNN) have been quite successful. CNNs use convolutional kernels to group pixels together, such that for a 100x100 pixel image, we do not need 10,000 weights to connect each pixel. These networks are designed to extract information from images, such as classification. This has become the de facto standard for image recognition in the past 10 years (Krizhevsky et al. 2012, Zhao et al. 2024)

Recently, another model has become prevalent: the transformer (Vaswani et al. 2023). A transformer takes tokens as inputs; in most cases, these tokens are words. The transformer can predict new tokens, leading to generative models. It was discovered that you can serialize an image into patches of 16x16 pixels and use those as tokens to train transformers successfully (Dosovitskiy et al. 2021). Initially outperforming "classical" CNNs, later the optimizations in the transformers have been backported to CNN (Liu et al. 2022). In this thesis, however, we will focus on transformers instead of CNNs, specifically the generation of tokens for transformers.

Ideally, we can find a set of tokens that describes an image, similar to how a set of words describes a sentence or text. While text tokens can encode grammatical or contextual information, image tokens should capture geometric information. Currently, the 16x16 patches do not contain any geometrical information. This is reintroduced in the embedding, but the way this is done is unstable and unpredictable for any transformation done on the tokens by the transformer. It also does not match the Euclidean geometry that we expect from images. This is also described in (Dosovitskiy et al. 2021), where we can see that the attention of a token is mostly on tokens that are on the same row/column axis. This is more likely because of the embedding, instead of the actual relevance of those tokens.

One way to generate such tokens is Gaussian splatting. This has been popularized in 2023 and is being used for 3D scene reconstruction (Kerbl et al. 2023). By taking multiple pictures and spawning Gaussian blobs in 3D to match each view, it results in a 3D scene represented by Gaussians. This is done using gradient descent to train the scenes to iteratively better match each image, using L1 and SSIM loss. Recently, Gaussian splatting has also been used in 2D for image representation (Zhang et al. 2025). This works essentially by taking only one image and training for that single image. This process works a lot simpler, as quite a few complications for 3D do not have to be made anymore. Besides encoding tokens for transformers, 2D Gaussian splatting can also be used in scenarios where compression is done once and decompression is done multiple times, such as textures for video games. Finally, 2D Gaussian splatting also has applications in super-resolution, i.e., upscaling images.

Tokenization of Gaussian parameters has already been studied (Dong et al. 2025). In this thesis, we aim to provide more geometric value to these tokens by using Lie theory. Furthermore, we hope to use more complex wavelets to encode more information in a single token. As each token has attention to all other tokens, transformer networks scale quadratically, limiting the number of tokens is thus key to smaller networks, limiting the needed memory and compute. We hope to use first and second order derivatives of the Gaussian function to define edges and lines more efficiently than a normal Gaussian can.

2 Theory

We are expanding the Gaussian splatting method with an explicit geometric description in terms of a Lie group, as well as using higher order derivatives to more efficiently encode edges and lines.

Concretely, we have some image $f : \mathbb{R}^2 \mapsto \mathbb{R}^3$ (or more precisely $f : [0, H] \times [0, W] \mapsto [0, 1]^3$) which we want to approximate with Gaussian functions. This representation will look like

$$f \approx \sum c_i e^{-\|x-y_i\|_A^2}. \tag{2.2}$$

This representation is already used for various applications such as superresolution (source), high-fps rendering, and data compression. Our goal is to create a sparse representation that holds geometric information, such that we can use it as image tokenization for transformer models.

To retain geometric information as well as possible, we will use Lie groups to position the Gaussians. The representation will then look similar to

$$f \approx \sum c_i \left(g_i \triangleright e^{-\|x\|^2}\right). \tag{2.4}$$

We can see that (2.4) is equivalent to (2.2) in terms of the goal it achieves, but the way we represent the parameters is key to retaining geometric information in the transformer.

2.1 Gaussian layers

Instead of using just the Gaussian function as is usual in Gaussian splatting, we will also use the first and second order derivatives of the Gaussian function. This is defined as

$$\begin{aligned} \phi_0(x) &:= e^{-\|x\|^2} \\ \hat{\phi}_1 &= \frac{\mathrm{d}}{\mathrm{d}x_1}\phi_0 \\ \hat{\phi}_2 &= \frac{\mathrm{d}}{\mathrm{d}x_1}\phi_1 \end{aligned} \tag{2.6}$$

Please note that the choice of variable (x_1 or x_2) for the derivative does not matter, as the axes are orthogonal and the function can be rotated by 90 deg to get the other derivative.

This will give us more complex features to work with. These three Gaussians will be layered in the same location. Below, we can see the derivatives when they are unscaled.

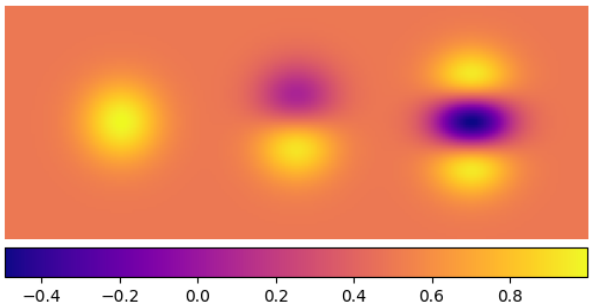


Figure 1: Unscaled (f.l.r.): Gaussian (ϕ_0), first-order derivative ($\hat{\phi}_1$) and second-order derivative ($\hat{\phi}_2$) of a Gaussian, defined in (2.6).

The idea is that the "zeroth-order" derivative is used as the base color, and that the derivatives are used as accents. The first-order derivative contains a fast drop-off, which could be used for creating a sharp edge, and the second-order derivative contains a sharp bump in the middle; this could be used as lines.

Layering these as is will not result in a good overlap, however, because the "support" of these functions differs a lot. We can scale the derivatives such that their "support" matches better with the Gaussian. Scaling the first-order by 0.7 and the second-order by 0.5, we get the following Gaussians, which will be layered to create more complex structures.

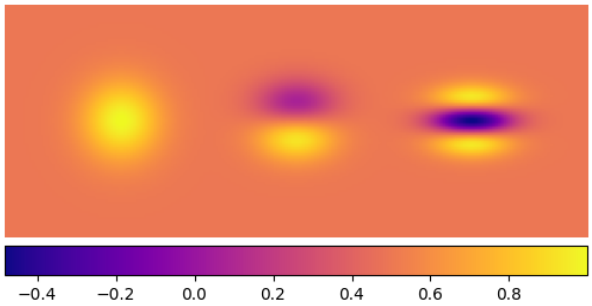


Figure 2: Scaled (f.l.r.): Gaussian (ϕ_0), first-order derivative (ϕ_1), and second-order derivative (ϕ_2) of a Gaussian.

Each of these Gaussians has an independent set of color values. $c^{i,j} \in \mathbb{R}$, $i \in \{0, 1, 2\}$ for each the Gaussian layers, and $j \in \{1, 2, 3\}$ for the color channels, totalling 9 values. Each color value is not directly the color that the Gaussian attains, but rather an addition or subtraction in a certain direction for the final color. For each pixel color $pc \in [0, 1]^3$, we start with a default of 0.5. This has the advantage that the image is color invertible by multiplying all the color values by -1, but also does not introduce an implicit bias to the color black, as opposed to white (or vice versa). Therefore, the color values of the Gaussians can attain any value in $\mathbb{R}$, however, most likely they will be pushed between $[-0.5, 0.5]^3$ as the input image contains only values between $[0, 1]^3$, and any overflow will be automatically clipped. Another way this can be done is by putting the final color value into a sigmoid function, both have similar performance.

Finally, the function in (2.4) will then be approximated by

$$f \approx \sum 0.5 + c_0^i(g_i \triangleright \phi_0) + c_1^i(g_i \triangleright \phi_1) + c_2^i(g_i \triangleright \phi_2).$$

2.2 Lie groups

The component for position, rotation, and scale should ideally be represented by a Lie group. As this means our representation can benefit from all the structure and tools that Lie groups provide. Specifically, we will look at Affine groups.

2.2.1 Affine group

A group G that covers all positions, rotations, and scaling is defined as follows.

Definition 2.2.1 (The affine group with positive determinants) The *2-dimensional, real affine group with positive determinants* is defined as

$$G = \mathrm{Aff}^+(\mathbb{R}^2) := \mathbb{R}^2 \rtimes \mathrm{GL}^+(\mathbb{R}^2)$$

with the group operation given by

$$(x_1, A_1)(x_2, A_2) := (A_1 x_2 + x_1, A_1 A_2)$$

for all $(x_1, A_1), (x_2, A_2) \in \mathrm{Aff}^+(\mathbb{R}^2)$.

Where $\mathrm{GL}^+(\mathbb{R}^2)$ is the group of 2-dimensional real linear transformations with positive determinants, i.e. 2×2 matrices with positive determinant. This positive determinant ensures that the transformation does not flip the image, as the Gaussians and their derivatives are symmetric, which is redundant. Furthermore, this also causes the group to consist of two separate components, which we cannot move between in a continuous fashion. Intuitively, you can rotate continuously, but you cannot flip an image continuously. Later, this will also make mathematical sense when we decompose $\mathrm{GL}^+(\mathbb{R}^2)$ into a rotational matrix and a positive definite symmetric matrix, which is only possible with a positive determinant.

Lemma 2.2.2 The unit element of G is $(0, I)$, the inverse of any $(x, A) \in G$ is $(-A^{-1}x, A^{-1})$ and the G has a dimension of 6.

Proof:

Let $(x, A) \in G$.

The unit element is $(0, I)$, as $(x, A)(0, I) = (A0 + x, AI) = (x, A)$
and $(0, I)(x, A) = (Ix + 0, IA) = (x, A)$.

The inverse is $(-A^{-1}x, A^{-1})$ as $(x, A)(-A^{-1}x, A^{-1}) = (-AA^{-1}x + x, AA^{-1}) = (0, I)$
and $(-A^{-1}x, A^{-1})(x, A) = (A^{-1}x - A^{-1}x, A^{-1}A) = (0, I)$.

The dimension is trivial.

■

Proposition 2.2.3 $\mathrm{Aff}^+(\mathbb{R}^2)$ is a *Lie group*.

Proof:

We will show that $(x, y) \mapsto x^{-1}y$ with $x, y \in G$ is a smooth map.

As matrix inverse and multiplication and scalar multiplication and addition are all smooth, we conclude that the map $(x, y) \mapsto x^{-1}y$ is also smooth.

■

The affine group acts on $\mathbb{R}^2$ in the following manner:

$$g \triangleright y = (x, A) \triangleright y := Ay + x$$

for all $y \in \mathbb{R}^2$ and $g = (x, A) \in \mathrm{Aff}^+(\mathbb{R}^2)$. Similarly, for subsets $S \subset \mathbb{R}^2$, we define

$$g \triangleright S := \{g \triangleright y \mid y \in S\}.$$

This allows us to use this Lie group to position Gaussians in our image. The next thing to consider is that the Gaussians we use to fill our image are not one point, but a collection of

points defined by a function in $\mathbb{R}^2$. Therefore, we will also need to define how this group acts on a function on $\mathbb{R}^2$. This is induced as follows:

Let $f : \mathbb{R}^2 \to \mathbb{R}$ then we define $g \triangleright f : \mathbb{R}^2 \to \mathbb{R}$ by

$$(g \triangleright f)(y) := f(g^{-1} \triangleright y).$$

2.2.2 Lie groups & parameter space

This works quite well for positioning the Gaussians with some particular translation, scaling, and rotation. However, to find the set of group elements such that this matches our target image, we will use gradient descent, as will be explained later. This requires the parameters to lie in a vector space. This is not the case by default for the Lie group. This means that when naively applying gradient descent on Aff^+ as a subset of $\mathbb{R}^6$, the parameters could no longer be in the Lie group after gradient descent steps. For example, the determinant could turn negative, leaving the group. This means that this parameter has no longer any useful meaning, and cannot be used to position the Gaussian.

For this reason, we will search for a parameter space, V, such that V is a vector space, and there exists a surjective function $\psi : V \to G$. As V is a vector space, we can apply gradient descent on V, use ψ to map V to G, and render the image using G.

2.2.3 The Lie algebra $\mathfrak{aff}(\mathbb{R}^2)$

To solve this issue, we would like to make use of the Lie algebra. This is the tangent space at the identity of the Lie group. The Lie algebra can be mapped into the Lie group (and vice versa), but is also a vector space.

The Lie algebra of $\mathrm{Aff}^+(\mathbb{R}^2)$ is $\mathfrak{aff}(\mathbb{R}^2) := \mathbb{R}^2 \rtimes \mathfrak{gl}(\mathbb{R}^2) \equiv \mathbb{R}^2 \rtimes \mathbb{R}^{2\times 2}$. Note that we say $\mathfrak{aff}$ and not $\mathfrak{aff}^+$; this is because $\mathfrak{aff}$ is the Lie algebra of both the Aff^+ and Aff Lie groups, illustrating that there is not a one-to-one relationship between Lie groups and Lie algebras.

Lemma 2.2.4 The Lie group $\mathrm{Aff}^+(\mathbb{R}^2)$ has a matrix representation, defined as

$$(x, A) \mapsto \begin{pmatrix} A & x \\ 0 & 1 \end{pmatrix}.$$

The Lie algebra $\mathfrak{aff}(\mathbb{R}^2)$ has a corresponding matrix representation, defined as

$$(v, W) \mapsto \begin{pmatrix} W & v \\ 0 & 0 \end{pmatrix}.$$

Proof:

We need to show that the representation of the Lie group preserves the group product.

Let $(x_1, A_1), (x_2, A_2) \in \mathrm{Aff}^+(\mathbb{R}^2)$.

$(x_1, A_1)(x_2, A_2)$ in matrix form is
$\begin{pmatrix} A_1 & x_1 \\ 0 & 1 \end{pmatrix}\begin{pmatrix} A_2 & x_2 \\ 0 & 1 \end{pmatrix} = \begin{pmatrix} A_1A_2 & A_1x_2+x_1 \\ 0 & 1 \end{pmatrix}$.

Converting this back is $(A_1x_2 + x_1, A_1A_2)$, which proves the matrix multiplication is equivalent to the group operation.

■

2.2.4 The exponential & logarithmic map

Lie groups and Lie algebras are related to each other through the exponential map. For our case the exponential map $\exp_{\mathrm{Aff}^+(\mathbb{R}^2)} : \mathfrak{aff}(\mathbb{R}^2) \to \mathrm{Aff}^+(\mathbb{R}^2)$ is given by exponent function.

Doing this for W and v seperately is equivalent to

$$\exp((v, W)) := \left(\left(\int_0^1 e^{tW} \, \mathrm{d}t \right) v, e^W \right). \tag{7}$$

Unfortunately, from (Culver 1966) we know that the Aff^+ exponential map is not surjective. So we cannot use $\mathfrak{aff}$ to create every possible Gaussian. Besides the fact that it is not surjective, the integral is also a problem. This means that the function is not a convenient closed formula, but instead must be computed numerically.

2.3 Kaji-Ochiai parameterization

Since using the default Lie algebra $\mathfrak{aff}$ from the Lie group Aff^+ as our parameter space will limit us a lot, we will need to look a bit further. We adapt the findings from (Kaji and Ochiai 2016), where they describe a parameterization of $\mathrm{Aff}^+(\mathbb{R}^3)$, where this parameterization has a similar role as the Lie algebra. This parameterization is for a 3D affine transformation; it starts from what is essentially the Lie algebra but constructs a different surjective mapping than the exponential onto the group (but still closely related). We will simplify this parametrization to 2 dimensions.

The Kaij-Ochiai parameterization works by factoring the Lie algebra of $\mathrm{Aff}^+(\mathbb{R}^2)$ as per

$$\mathfrak{aff}(\mathbb{R}^2) \equiv \mathbb{R}^2 \times \mathfrak{so}(2) \times \mathfrak{sym}(2) \equiv \mathbb{R}^2 \times \mathbb{R} \times \mathbb{R}^3 \equiv \mathbb{R}^6$$

where $\mathbb{R}^2$ encodes the translation generator, $\mathfrak{so}(2)$ the rotation generator and $\mathfrak{sym}(2)$ the scale and shearing generator. Here $\mathfrak{so}(2)$ is the Lie algebra of the rotation group $\mathrm{SO}(2)$ and is given by the real 2×2 anti-symmetric matrices, and so has only a single degree of freedom:

$$\mathfrak{so}(2) = \left\{ \begin{pmatrix} 0 & -r \\ r & 0 \end{pmatrix} \middle| \, r \in \mathbb{R} \right\}.$$

The second vector space $\mathfrak{sym}(2)$ is the set of (real) symmetric 2×2 matrices, but it is not the Lie algebra of any Lie group since the Lie bracket of two symmetric matrices is not necessarily a symmetric matrix; it is anti-symmetric in fact:

$$[A, B]^T = (AB - BA)^T = B^T A^T - A^T B^T = -(A^T B^T - B^T A^T) = -[A, B]$$

for all $A, B \in \mathfrak{sym}(2)$.

This is clearly the case with the example $A = \left(\begin{smallmatrix} 1 & 2 \\ 2 & 3 \end{smallmatrix}\right)$ and $B = \left(\begin{smallmatrix} 3 & 4 \\ 4 & 5 \end{smallmatrix}\right)$. Then

$$[A, B]^T = \begin{pmatrix} 11 & 14 \\ 18 & 23 \end{pmatrix} - \begin{pmatrix} 11 & 18 \\ 14 & 23 \end{pmatrix} = \begin{pmatrix} 0 & -4 \\ 4 & 0 \end{pmatrix} = -[A, B] \neq [A, B].$$

The vector space $\mathfrak{sym}(2)$ has three degrees of freedom:

$$\mathfrak{sym}(2) = \left\{ \begin{pmatrix} s_1 & s_3 \\ s_3 & s_2 \end{pmatrix} \middle| \, s_1, s_2, s_3 \in \mathbb{R} \right\}.$$

Definition 2.3.1 The parametrization map $\phi : \mathbb{R}^2 \times \mathbb{R} \times \mathbb{R}^3 \to \mathrm{Aff}^+(\mathbb{R}^2)$ is given by

$$\phi(x, r, s) := \left(x, \exp\begin{pmatrix} 0 & -r \\ r & 0 \end{pmatrix} \exp\begin{pmatrix} s_1 & s_3 \\ s_3 & s_2 \end{pmatrix}\right). \tag{8}$$

The matrix exponentials in the parametrization map 8 have the following closed formulae

$$\exp_{\mathfrak{sym}}(r) = \exp\begin{pmatrix} 0 & -r \\ r & 0 \end{pmatrix} = \begin{pmatrix} \cos r & -\sin r \\ \sin r & \cos r \end{pmatrix}$$

and

$$\begin{aligned}\exp_{\mathfrak{so}}(s_1, s_2, s_3) = \exp\begin{pmatrix} s_1 & s_3 \\ s_3 & s_2 \end{pmatrix} &= e^{\frac{s_1+s_2}{2}} \begin{pmatrix} \cosh(a) + \frac{s_1-s_2}{2a}\sinh(a) & \frac{s_3}{a}\sinh(a) \\ \frac{s_3}{a}\sinh(a) & \cosh(a) + \frac{s_2-s_1}{2a}\sinh(a) \end{pmatrix} \\ &= e^{\frac{s_1+s_2}{2}} \left(\cosh(a)\begin{pmatrix} 1 & 0 \\ 0 & 1 \end{pmatrix} + \frac{\sinh(a)}{a}\begin{pmatrix} \frac{s_1-s_2}{2} & s_3 \\ s_3 & \frac{s_2-s_1}{2} \end{pmatrix}\right)\end{aligned}$$

where $a := \frac{1}{2}\sqrt{(s_1 - s_2)^2 + 4s_3^2}$.
Note that $\det\left(\exp\begin{pmatrix} 0 & -r \\ r & 0 \end{pmatrix}\right) = 1$ and $\det\left(\exp\begin{pmatrix} s_1 & s_3 \\ s_3 & s_2 \end{pmatrix}\right) = e^{s_1+s_2} > 0$.

Furthermore, we can also look at the inverse of the parameterization.

Definition 2.3.2 A locally differentiable inverse $\overleftarrow{\phi} : \mathrm{Aff}^+(\mathbb{R}^2) \to \mathbb{R}^2 \times \mathfrak{so}(2) \times \mathfrak{sym}(2)$ is given by

$$\overleftarrow{\phi}(x, A) := \left(x, \log\left(A(A^TA)^{-\frac{1}{2}}\right), \log\left(\sqrt{A^TA}\right)\right). \tag{9}$$

This local inverse relies on the *polar decomposition* of a square matrix (Hall 2015, § 2.5). Every square matrix A admits a polar decomposition into an orthogonal matrix R and a positive semi-definite matrix S so that $A = RS$. If A is invertible, which it is in our case, then the polar decomposition is unique and S is positive definite. The matrices R and S are calculated as per

$$R = A(A^TA)^{-\frac{1}{2}} \quad \text{and} \quad S = (A^TA)^{\frac{1}{2}}.$$

Since A is invertible A^TA is symmetric positive definite and so has a unique square root, which is itself symmetric positive definite and hence uniquely invertible. Clearly $A = RS$ and S is symmetric positive definite, orthogonality of R follows from

$$R^TR = \left(A(A^TA)^{-\frac{1}{2}}\right)^T\left(A(A^TA)^{-\frac{1}{2}}\right) = (A^TA)^{-\frac{1}{2}}(A^TA)(A^TA)^{-\frac{1}{2}} = I.$$

2.4 A note on shearing

Shearing is the changing of the angle of the orthogonal base vectors of the space. No shearing with respect to $\{e_1, e_2\}$ implies that $(Ae_1)^TAe_2 = 0$. In our case $e_1 := (1, 0)^T$ and $e_2 := (0, 1)^T$. This shearing can have some interesting properties, so we will take a closer look to see if we want this or if we want to exclude this from the Gaussian transformations.

Dissecting the polar decomposition from earlier, we can see that $R \in \mathrm{SO}(2)$ does rotation, and $S \in \mathrm{SPD}(2)$ does scaling and this shearing.

We define S as

$$S = \begin{pmatrix} s_1 & s_3 \\ s_3 & s_2 \end{pmatrix}.$$

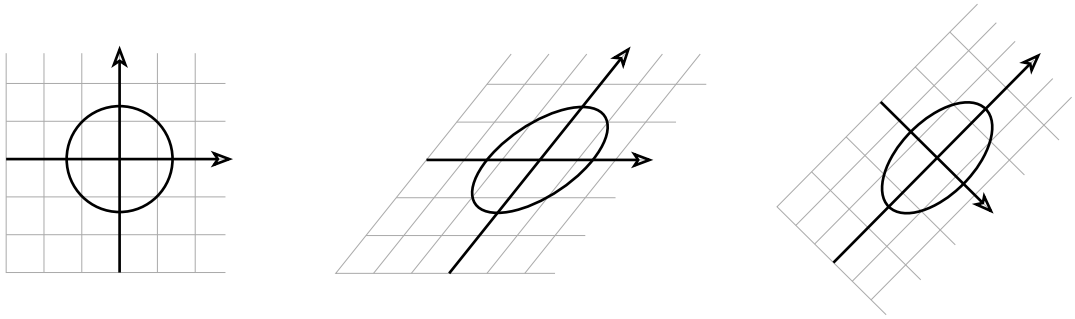


Figure 3: Visualization of shearing. Left is not sheared, middle is sheared, right is scaled and then rotated

Then we can see that s_3 generates shearing, as when $s_3 = 0$ then we have $(Se_1)^T(Se_2) = (s_1, 0)(0, s_2)^T = 0$. And vice versa, if $s_3 \neq 0$, then $(Se_1)^T(Se_2) = (s_1, s_3)(s_3, s_2)^T = s_1 s_3 + s_3 s_2 \neq 0$ as $s_1, s_2 > 0$, clearly introducing a shear.

We know that the stabilizer of ϕ_0 is $\mathrm{SO}(2)$. This makes sense as rotating an unscaled Gaussian while giving us that exact same Gaussian. I.e.

$$\mathrm{Stab}_G(\phi_0) := \{g \in G \mid g \rhd \phi_0 = \phi_0\} \equiv \mathrm{SO}(2)$$

Furthermore, we can see that the effect of a shear on a Gaussian can also be generated by a rotation and a scale. I.e., we can remove s_3 and still generate all Gaussian configurations.

Theorem 2.4.1 (Shear generation)

For all $A \in \mathrm{GL}^+(\mathbb{R}^2)$, there exists a $\overline{R} \in \mathrm{SO}(2)$ and $D \in \mathrm{SPD}(2)$ which is also diagonal, i.e. $D = \begin{pmatrix} e^{s_1} & 0 \\ 0 & e^{s_2} \end{pmatrix}$, such that $A \rhd \phi_0 = \overline{R}D \rhd \phi_0$.

Proof:

We know that $A = RS$, then we can do an eigendecomposition and decompose S as

$$S = Q\Lambda Q^T.$$

Where $\Lambda = \begin{pmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{pmatrix}$ with $\lambda_{\{1,2\}}$ as the eigenvalues from S. $Q \in O(2)$, however we can flip the eigenvalues in the decomposition when $\det(Q) = -1$, therefore without loss of generality $Q \in \mathrm{SO}(2)$.

This means that we get the following.

$$A \rhd \phi_0 = RS \rhd \phi_0 = RQ\Lambda Q^T \rhd \phi_0 = RQ\Lambda \rhd (Q^T \rhd \phi_0) = RQ\Lambda \rhd \phi_0.$$

Now we can define $\overline{R} := RQ \in \mathrm{SO}(2)$ and $D := \Lambda$.

Thus $A \rhd \phi_0 = \overline{R}D \rhd \phi_0$. ■

Proposition 2.4.2 (Polar decomposition ordering)

Reversing rotational and scaling in the parametrization of Kaij-Ochiai will remove the geometric interpretation of s_3 as the amount of shearing with respect to $\{e_1, e_2\}$. I.e.

$$A(r, s_1, s_2, 0) = \exp\begin{pmatrix} 0 & -r \\ r & 0 \end{pmatrix} \exp\begin{pmatrix} s_1 & 0 \\ 0 & s_2 \end{pmatrix}$$

never contains no shearing in $\{e_1, e_2\}$.

While

$$B(r, s_1, s_2, 0) = \exp\begin{pmatrix} s_1 & 0 \\ 0 & s_2 \end{pmatrix} \exp\begin{pmatrix} 0 & -r \\ r & 0 \end{pmatrix}$$

can contain shearing in $\{e_1, e_2\}$.

Proof:

We will show that any transformation $A = RS$ with

$$R = \begin{pmatrix} \cos r & -\sin r \\ \sin r & \cos r \end{pmatrix}$$

and

$$S = \begin{pmatrix} \exp(s_1) & 0 \\ 0 & \exp(s_2) \end{pmatrix}$$

applied as RS results in $e_1 e_2^T = 0$.

We can see this by

$$\begin{aligned} (Ae_1)^T(Ae_2) &= (RSe_1)^T(RSe_2) \\ &= e_1^T S^T R^T RSe_2 \\ &= e_1^T S^T Se_2 \qquad \text{As R is orthogonal} \\ &= (s_1, 0)^T(0, s_2) \\ &= 0. \end{aligned}$$

It is also true that for $B = SR$, applying SR in the other order can result in shearing; we will show this with an example.

Define

$$S = \begin{pmatrix} 1 & 0 \\ 0 & 2 \end{pmatrix}$$

and

$$R = \begin{pmatrix} \frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \\ -\frac{\sqrt{2}}{2} & \frac{\sqrt{2}}{2} \end{pmatrix}.$$

Now clearly $(SRe_1)^T(SRe_2) = -\frac{3}{2} \neq 0$

■

And thus we can see that the parametrization is still surjective with $s_3 = 0$ for 0-th order Gaussians and other radially symmetric wavelets. We can therefore remove this in the parameter space $V \equiv \mathbb{R}^5$. We can still map this using ψ, however, this will no longer map to Aff^+, but a subset of that where there exists no shearing. This subset is not a group.

Corollary 2.4.2.1

Define $T \subset \mathrm{GL}^+(\mathbb{R}^2)$ as $T := \left\{A \in \mathrm{GL}^+(\mathbb{R}^2) \mid (Ae_1)^T Ae_2 = 0\right\}$, then T is not a group.

Intuitively, the previous results make sense as the rotational symmetry of ϕ_0 makes it such that the basis choice $\{e_1, e_2\}$ is arbitrary and has no dependency on anything else. For ϕ_1 and ϕ_2, this choice is relevant, as we already take the derivative in one of these directions. Shearing will add a unique property to these. Below is an example of the concept of shear in ϕ_0 being generated without s_3 or fully with s_3, and the effects this has on ϕ_1 and ϕ_2.

Figure 4: Left: Shearing via scale & rotate, $s_3 = 0$ | Right: Shearing via setting s_3
We notice that shearing only has effects on ϕ_1 and ϕ_2.

While this is not exactly the same, and for $\phi_{\{1,2\}}$ parameterization with $s_3 = 0$ is no longer surjective, however, removing this property would give us some better analysis on the anisotropy[1] of a Gaussian, as the anisotropy is now uniquely generated by the scaling parameters.

3 Methods

We will look at the methods that we will be using to actually create the Gaussian representations.

3.1 Rendering function

The target image we want to approximate is defined as $f(x) : \mathbb{R}^2 \to \mathbb{R}^3$.

[1] Anisotropy is how "stretched" an ellipsoid, or in our case a Gaussian, is. We will see later why this is useful

The final image that is generated from the parameters is defined by

$$\hat{f}(x) = \sum_{k=0}^{n} \sum_{i}^{3} c_k^i (g_k \triangleright \phi)(x) = \sum_{k=0}^{n} \sum_{i}^{3} c_k^i (\psi(v_k) \triangleright \phi)(x).$$

Where $c_k^i \in \mathbb{R}^3$ and $g_k \in G$.

To find the optimal parameters c and v, we will use gradient descent as explained earlier. For this, we will define a score, called a loss, that determines how well a set of parameters represents the image. Taking steps downwards of the gradient will find us a local minimum. Using extra methods such as optimizers, a scheduler, and modifying the loss, we can further increase the quality of the minimum that was found.

3.2 Loss

The loss function consists of a few different components. Most importantly, the L_1 loss, formally defined as $\|f - \hat{f}\|$. Besides this, we will also use the structural similarity index measure (SSIM) (Wang et al. 2004). These will be weighed with λ. The total loss for how well the image looks will be $L_{\text{image}} = \lambda\|f - \hat{f}\| + (1-\lambda) * \left(1 - \text{SSIM}\left(f, \hat{f}\right)\right)$. The L_1 loss has a big focus on individual pixel differences. While this is useful, it might overfit on certain pixels. SSIM, on the other hand, focuses on differences on a structural level, looking more at luminance and contrast. Combining these two has been done before in similar image representation projects, and work here again (Kerbl et al. 2023). Later in Section 4.2, this loss will expanded upon to create a final loss.

3.3 Culling

As we want to decrease the number of Gaussians that we save, we will remove the Gaussians based on their parameters. This process is called culling (Kerbl et al. 2023). The most obvious is that we would like to remove Gaussians where the color values are negligibly small for all Gaussian layers. Formalized by

$$\sum_{j}^{3} \left|c^{1,j}\right| \leq 0.05 \wedge \sum_{j}^{3} \left|c^{2,j}\right| \leq 0.05 \wedge \sum_{j}^{3} \left|c^{3,j}\right| \leq 0.05.$$

The other criterion is the size of the Gaussian. Formally, we do this by removing all Gaussians where

$$s1 + s2 \leq -7.5.$$

The problem is that these Gaussians are still visible, so doing this culling after all training is done could leave some gaps in the image. Therefore, we do this on 80% of the training, thus giving the gradient descent still some iterations to recover and fill the gaps.

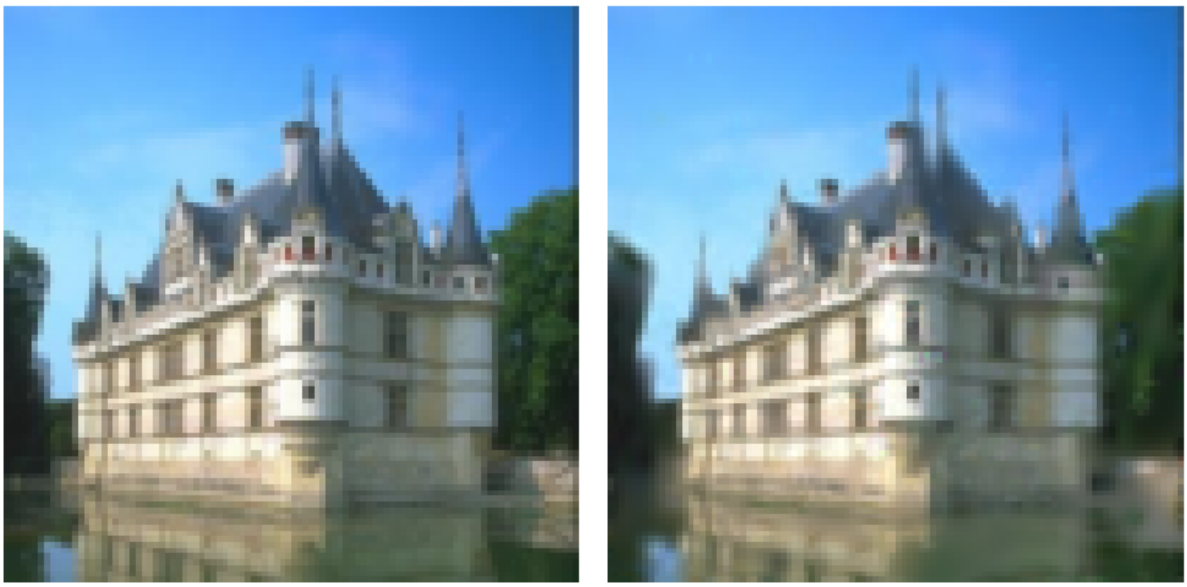
4 Results

To put this into practice, we use pytorch to make use of CUDA kernels and speed up the rendering. The source code is available on GitHub[2]. The images are constrained to 1:1 to ease the implementation; the theory can be easily scaled to any ratio. The resolution can differ, depending on the input. For the experiments, mostly 100x100 resolutions are used, as this is a limitation of the physical hardware on which the tests were run. Higher resolution images would use more VRAM.

[2]https://github.com/SuperVK/sparse-image-repr-bep

The hardware used is an `Intel(R) Core(TM) i7-10750H (12) @ z` and `NVIDIA Quadro T1000 Mobile`; on this hardware, training took between 1 minute and 5 minutes. However, minimising the training time is not a goal of the project.

In this section, we will discuss the results and adaptations we have made based on the findings. We will first present the final product as seen in Figure 5, which includes all the adaptations, which we will use as our base performance to compare against.



(a): Original castle image (b): Final performance of the representation

Figure 5: Comparing the original image against the final representation. The final representation uses 1500 Gaussians.

4.1 Initialization

For the initialization, we use a hyperparameter for the number of Gaussians that will be spawned. The color parameters of ϕ_0 will be initialized with a normal distribution $N(0,1)$, for ϕ_1 and ϕ_2, this is $N(0,0.1)$. I.e. $c_k^{0j} \sim N(0,1)$ and $c_k^{ij} \sim N(0,1)$ for $i \in \{1,2\}$. This prevents the first and second orders from being too active. The position of the Gaussians is also initialized with $N(0,1)$. The rotational parameter r is generated between 0 and 2π uniformely. The scaling parameters are also generated with $N(0,1)$ but immediately shifted down by -4, in other words $s_1, s_2 \sim N(-4,1)$. If this is not done, the Gaussians will overlap too much and fight against each other, resulting in streaks. This can be seen in Figure 6.

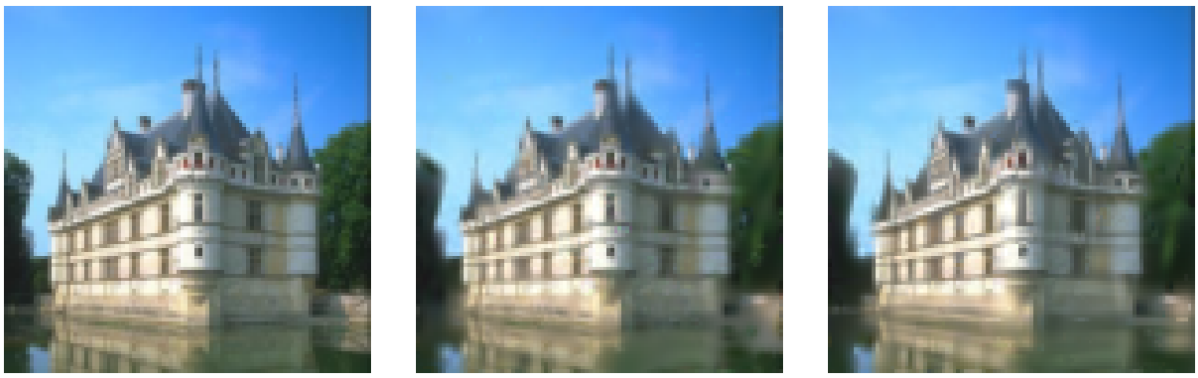
(a): Original castle image (b): Final performance of the representation (c): Representation with unscaled initialization

Figure 6: Comparing the different initialization techniques. Both representations use 1500 Gaussians, 100x100.

4.2 Artifacts

Besides the actual representation, we have two other things that we want to discourage to prevent the generation of invisible or unwanted artifacts. As gradient descent tries to find the minimum of the loss, we can put things in the loss to discourage certain behavior. First of all, we want to limit the anisotropy, to make sure Gaussians do not become stretched out, but stay fairly round. Furthermore, we also do not want Gaussians to become too small; to be exact, they must not be smaller than a pixel, as this will make them hidden while rendering, but they still contain information in the final embedding. Both of these are already being prevented in Section 3.3, culling, but to push Gaussians in the right direction before fully being removed, we also want to put this in the loss.

The effects of small Gaussians are not directly apparent; seemingly, the performance is roughly similar.



(a): Original castle image (b): Final performance of the representation. SSIM: 0.0418 (c): No sizing in loss, no culling. SSIM: 0.0471

Figure 7: Comparing the size penalty, both representations start with 1500 Gaussians, 100x100.

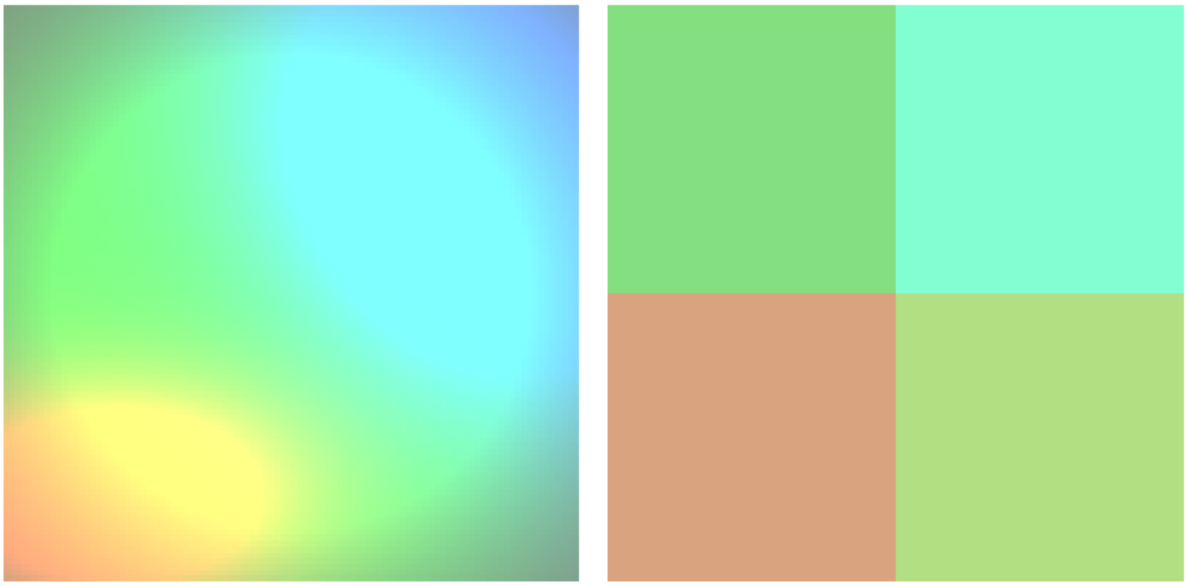
However, when rendering the representations upscaled, the difference becomes apparent. Both images are trained in a 100x100 resolution, but the final render is upscaled to 150x150.

(a): Final performance of the representation, upscaled to 150x150.

(b): No sizing in loss, upscaled to 150x150.

Figure 8: Comparing the upscaled renders of the representation. Not penalizing the size of the Gaussian clearly creates hidden artifacts

You will see the artifacts in the right image. These are not visible in the 100x100, as they are smaller than a pixel in the 100x100 render. This comes from the fact that the representation is infinitely detailed, but the rendering is a sampling of that representation. For each pixel, instead of the average over the whole pixel, only a specific spot within the pixel is picked. Taking the average is computationally expensive and only shifts the same problem to a smaller level. These small Gaussians do influence the sample spots of the pixels, so removing them is not an option. This effect is also showcased in Figure 9.



(a): 100x100

(b): 2x2

Figure 9: The same Gaussian representation, sampled differently. In this implementation, the sampling happens at the bottom left corner. The bright yellow spot is not visible anymore.

The problem with these subpixel Gaussians is that they will still contain information in the tokens. To keep these tokens pure and as close to the original image as possible, we want to remove this.

To already penalize this in the training process, we will add the following loss function $L_{\text{sizing}} = \overline{\exp(-(s_1 + s_2) - 8)} * \lambda_{\text{sizing}}$ to the loss. Where λ_{sizing} determines how much influence this loss function has.

The effect of anisotropic Gaussians is fairly similar, as can be seen below.

(a): Original castle image

(b): Representation without shearing loss, and no culling at 100x100.

(c): Representation without shearing loss, and no culling, upscaled to 150x150.

Figure 10: The same Gaussian representation, where shearing is not penalized in the loss. In the upscaled version, the artifacts are clearly visible. The SSIM is 0.0396

Here we define the anisotropy as $L_{\text{anisotropy}} = \overline{|s_1 + s_2|} * \lambda_{\text{anisotropy}}$. Because shear is exclusively generated by the size, this is an easy way to penalize for this.

The final loss is then defined as

$$\begin{aligned} L &= L_{\text{image}} + L_{\text{anisotropy}} + L_{\text{sizing}} \\ &= \lambda \|f - \hat{f}\| + (1 - \lambda) * \left(1 - \text{SSIM}\left(f, \hat{f}\right)\right) + \overline{|s_1 + s_2|} * \lambda_{\text{anisotropy}} + \overline{\exp(-(s_1 + s_2) - 8)} * \lambda_{\text{sizing}} \end{aligned}$$

with $\lambda_{\text{anisotropy}}$ and λ_{sizing} are both set to 0.1, λ is set to 0.9.

As $\mathbb{R}^2 \times \mathbb{R} \times \mathbb{R}^3$ are all vector spaces, and the loss function is continuous and differentiable, we can use gradient descent to optimize this. We will use AdamW and a scheduler to further optimize the training process.

4.3 Number of Gaussians

The number of Gaussians has a significant impact on both the quality of the image and the amount of tokens, but we want to have the best quality while having the least amount of tokens. Finding an optimal is therefore essential, as too many tokens will slow down the training of the model, but quality might be needed to train the transformer efficiently. This amount is also highly dependent on the resolution of the original image and the nature of the image. Images with a lot of contrast and high-frequency data, such as text in an image, require more Gaussians.

The final image we have been using uses 1500 Gaussians for 100x100. Below are the effects of doubling and halving that

(a): Original castle image

(b): Representation with 750 Gaussians initialized
SSIM Loss: 0.0770
L1 Loss: 0.0186

(c): Representation with 1500 Gaussians initialized
SSIM Loss: 0.0429
L1 Loss: 0.0131

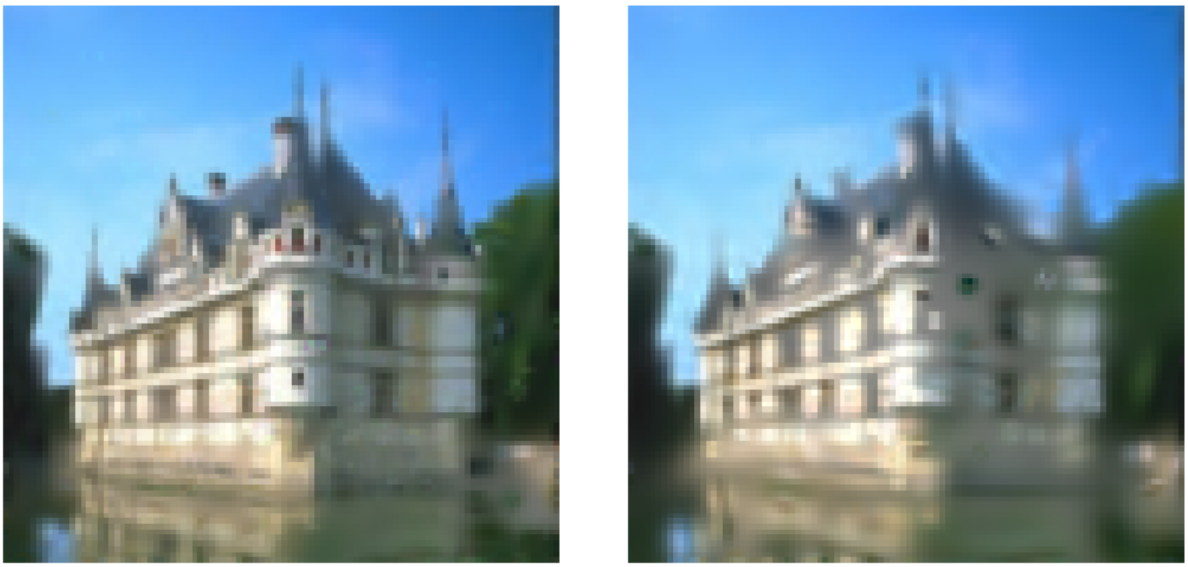
(d): Representation with 3000 Gaussians initialized
SSIM Loss: 0.0088
L1 Loss: 0.0056

Figure 11: Comparing different starting amounts of Gaussians.

Clearly, the progression is visible; the representation initialized with 3000 Gaussians is almost identical to the naked eye. An important side note here is that the tokens grow exponentially in terms of the cost of the neural network.

4.4 Culling

In Section 3.3, we already talked about how we will do the culling. We will also take a short look at how this works in practice. One of the big things that makes a difference is penalizing the size of the loss. If we do not do this, we get significantly worse results, and also a big reduction in Gaussians Figure 12.



(a): Final performance of the representation. Starts with 1500 Gaussians, ends with 1417 Gaussians after culling.

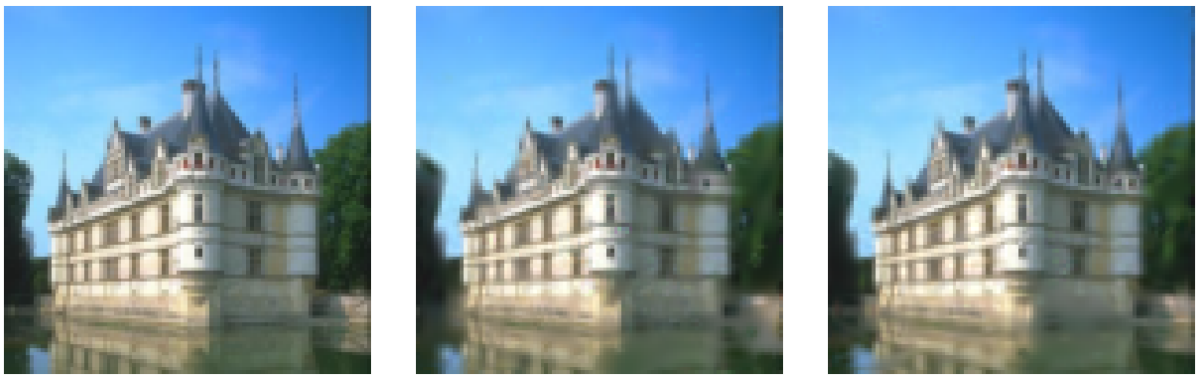
(b): Representation with culling, but no size in loss. Starts with 1500 Gaussians, ends with 632 Gaussians.

Figure 12: Not penalizing sizing in the loss results in a worse performance after culling. This makes sense as the representation will count too much on small Gaussians during training, which will be removed afterwards. This is especially noticeable for the gray areas, where there is a lack of Gaussians providing any color.

Furthermore, going back to Section 4.3, we can compare the start and end Gaussians.

	Start Gaussians	End Gaussians
Low amount:	750	731
Normal amount:	1500	1417
High amount:	3000	2827

About 5% of the Gaussians are being removed. Almost all of these Gaussians are being removed based on color, i.e., they provide little to nothing to the color values.



(a): Original castle image

(b): Final performance representation **SSIM:** 0.0418

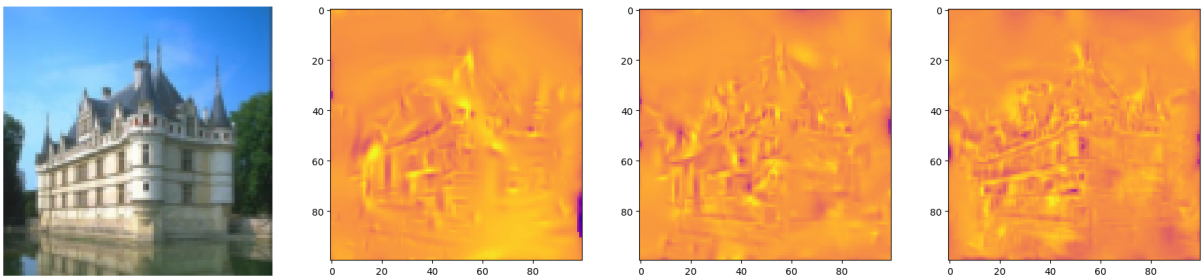
(c): Final performance representation, but without culling **SSIM:** 0.0350

Figure 13: The same Gaussian representation, where shearing is not penalized in the loss. In the upscaled version, the artifacts are clearly visible. The SSIM is 0.0396

While the performance is better in terms of the SSIM, we see the same issues as in Section 4.2.

4.5 Higher order Gaussians

To see the performance and the influence of the higher order Gaussians, we remove the ϕ_0 and only render $\phi_{\{1,2\}}$. Looking at the previous comparison, we can clearly see the lines and edges in the image.



(a): Original castle image

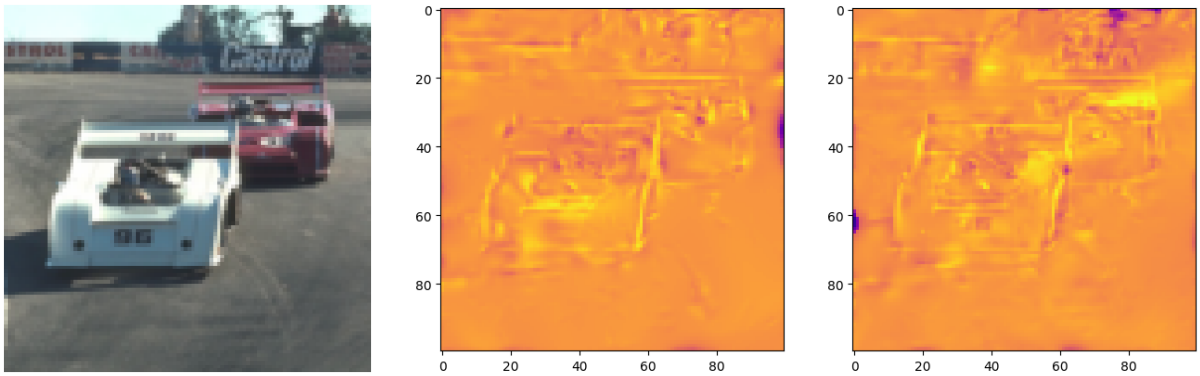
(b): Accents with 750 Gaussians initialized

(c): Accents with 1500 Gaussians initialized

(d): Accents with 3000 Gaussians initialized

Figure 14: Comparing the accents of different amounts of Gaussians. The lines and edges of the castle are more visible with more Gaussians.

You can clearly see where the image has more fidelity, and see the outlines of the castle. This effect is a lot stronger with more Gaussians.



(a): Original castle image (b): Accents with 1500 Gaussians initialized (c): Accents with 3000 Gaussians initialized

Figure 15: Comparing the accents of different amounts of Gaussians. The lines and edges of the cars and the curb are more visible with more Gaussians.

In the car's image containing more edges, this effect is more clearly visible.

5 Conclusion

In this thesis, we have looked at an alternative tokenization of images for transformer models using Lie theory and Gaussian splatting. We noticed that Aff^+ provides a good geometric representation of Gaussians. Unfortunately, the canonical vector space parameterization of Lie groups, Lie algebras, is not suitable. Therefore, we have derived an alternative parametrization that can be better used.

We have also analyzed the effect of shearing on the Gaussians to better analyze the training process. Noticing that the effect of shearing on Gaussians can also be generated by rotation and scaling. This leaves only two sizing parameters, which are geometrically easy to interpret, allowing us to penalize this metric.

Furthermore, more complex wavelets can be used to encode more complex features in images. Specifically, we can use first and second-order derivatives of Gaussians to encode edges and lines in an image. In theory, this should be able to reduce the number of Gaussians used in the representation, although this is outside of the scope of this thesis.

Finally, we can use metrics of Gaussians to ensure the generated tokens actually provide useful results. Specifically, we can make sure that the training process does not abuse the implementation details of the rendering function, such as Gaussians that are subpixel in size. Furthermore, we can also remove any Gaussians that are not relevant to the final representation, reducing the number of final Gaussians that are used as tokens.

6 Future work

This thesis has mostly been exploratory on various techniques and topics; ideally, we could better compare the actual impact on a large dataset of images. For this, more computing power is needed. This would also allow us to compare the image and performance at a higher resolution.

Furthermore, the wavelets have been chosen somewhat arbitrarily; perhaps there are other definitions of wavelets that can do a better representation. This will probably depend heavily

on the type of application. Where specific wavelets might be better suited for specific types of images.

Finally, the representation is ideally tested as tokens in a transformer to see if the transformer can properly work with our representation. Ideally, this is compared against a representation that does not include first and second order derivatives. We suspect that the transformer is able to extract extra information from the higher orders.

Bibliography

Culver WJ (1966) On the existence and uniqueness of the real logarithm of a matrix. Proceedings of the American Mathematical Society 17:1146–1151

Dong J, Wang C, Zheng W, et al (2025) GaussianToken: An Effective Image Tokenizer with 2D Gaussian Splatting. http://arxiv.org/abs/2501.15619. Accessed 10 July 2025

Dosovitskiy A, Beyer L, Kolesnikov A, et al (2021) An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale. http://arxiv.org/abs/2010.11929. Accessed 12 Jan 2023

Hall BC (2015) Lie Groups, Lie Algebras, and Representations: An Elementary Introduction. Springer International Publishing, Cham

Kaji S, Ochiai H (2016) A Concise Parametrization of Affine Transformation. SIAM Journal on Imaging Sciences 9:1355–1373. https://doi.org/10.1137/16M1056936

Kerbl B, Kopanas G, Leimkühler T, Drettakis G (2023) 3d gaussian splatting for real-time radiance field rendering.. ACM Trans Graph 42:139–131

Krizhevsky A, Sutskever I, Hinton GE (2012) ImageNet Classification with Deep Convolutional Neural Networks. In: Pereira F, Burges C, Bottou L, Weinberger K (eds) Advances in Neural Information Processing Systems. Curran Associates, Inc., p

Liu Z, Mao H, Wu C-Y, et al (2022) A ConvNet for the 2020s. https://arxiv.org/abs/2201.03545

Vaswani A, Shazeer N, Parmar N, et al (2023) Attention Is All You Need. https://arxiv.org/abs/1706.03762

Wang Z, Bovik A, Sheikh H, Simoncelli E (2004) Image quality assessment: from error visibility to structural similarity. IEEE Transactions on Image Processing 13:600–612. https://doi.org/10.1109/TIP.2003.819861

Zhang X, Ge X, Xu T, et al (2025) GaussianImage: 1000 FPS Image Representation and Compression by 2D Gaussian Splatting. In: Leonardis A, Ricci E, Roth S, et al. (eds) Computer Vision – ECCV 2024. Springer Nature Switzerland, Cham, pp 327–345

Zhao X, Wang L, Zhang Y, et al (2024) A review of convolutional neural networks in computer vision. Artificial Intelligence Review 57:. https://doi.org/10.1007/s10462-024-10721-6